

Introducción a la Programación con Python

Semana 4: Colecciones de Datos (Listas, Tuplas y Diccionarios)

∇Nablabs

Contenido

- 1 Introducción a las Colecciones
- 2 Listas
- 3 Tuplas
- 4 Diccionarios
- 5 Resumen Comparativo
- 6 Proyecto Semanal

¿Qué son las colecciones en Python?

Definición

Una estructura de datos (o colección) permite almacenar múltiples elementos dentro de una sola variable de manera organizada.

En Python existen cuatro tipos integrados principales, cada uno con características únicas:

 **Listas:** Ordenadas, mutables y permiten duplicados.

 **Tuplas:** Ordenadas, **inmutables** y permiten duplicados.

 **Diccionarios:** Desordenados (indexados por clave), mutables y claves únicas.

 **Sets (Conjuntos):** Desordenados, no indexados y elementos únicos.

¿Qué es una lista?

Es una secuencia ordenada de elementos encerrados entre corchetes []. Son mutables (se pueden modificar).

```
1 # Crear una lista de frutas
2 frutas = ["manzana", "banana", "cereza"]
3 print(frutas)
4
5 # Acceder mediante índices (empiezan en 0)
6 print(frutas[0]) # Salida: manzana
7 print(frutas[-1]) # Salida: cereza (ultimo elemento)
```

💡 Las listas pueden contener elementos de diferentes tipos (enteros, strings, booleanos, e incluso otras listas).

Modificación y Métodos de Listas

```
1 frutas = ["manzana", "banana"]
2
3 # Agregar un elemento al final
4 frutas.append("naranja")
5
6 # Insertar en una posición específica
7 frutas.insert(1, "uva")
8
9 # Eliminar un elemento por su valor
10 frutas.remove("banana")
11
12 print(frutas)
```

➔ Salida

```
['manzana', 'uva', 'naranja']
```

Slicing (Sublistas) ✂

Slicing

Permite extraer porciones de una lista usando la sintaxis `lista[inicio:fin:paso]`. El límite superior no se incluye.

```
1 numeros = [0, 1, 2, 3, 4, 5]
2
3 print(numeros[1:4])    # índices del 1 al 3 -> [1, 2, 3]
4 print(numeros[:3])     # desde el inicio hasta el 2 -> [0,
5                        1, 2]
6 print(numeros[3:])     # desde el índice 3 al final -> [3,
                        4, 5]
```

i El slicing es una herramienta muy potente y común para filtrar datos rápidamente.

¿Qué es una tupla?

Es una colección de elementos ordenada que se define usando paréntesis (). Su característica principal es que son **inmutables** (no se pueden modificar una vez creadas).

```
1 dimensiones = (1920, 1080)
2 print(dimensiones[0]) # Salida: 1920
3
4 # Intentar modificar una tupla arroja un error:
5 # dimensiones[0] = 1280
```

 Error si se intenta modificar

TypeError: 'tuple' object does not support item assignment

¿Por qué usar tuplas? ?

Ventajas de las Tuplas

Seguridad de datos: Ideal para proteger información que no debe cambiar (coordenadas, configuraciones).



Rendimiento: Son más rápidas en memoria que las listas.



Claves de diccionarios: Al ser inmutables, se pueden usar como claves en diccionarios (las listas no).

```
1 # Desempaquetado de tuplas (Unpacking)
2 punto = (4, 5)
3 x, y = punto
4 print(f"X: {x}, Y: {y}")
```


¿Qué es un diccionario?

Asocia pares de elementos clave: valor usando llaves `{}`. No se accede a los datos por índice, sino por su clave única.

```
1 carro = {  
2     "marca": "Toyota",  
3     "modelo": "Corolla",  
4     "anio": 2022  
5 }  
6  
7 # Acceder al valor mediante su clave  
8 print(carro["marca"]) # Salida: Toyota
```

⚠ Si intentas acceder a una clave que no existe, Python arrojará un `KeyError`. Puedes usar `carro.get("precio")` para evitarlo.

Operaciones con Diccionarios

```
1 usuario = {"nombre": "Ana", "edad": 25}
2
3 # Agregar o modificar un elemento
4 usuario["ciudad"] = "Bogota" # Crea nueva clave
5 usuario["edad"] = 26          # Actualiza clave existente
6
7 # Eliminar un elemento
8 del usuario["nombre"]
9
10 print(usuario)
```

➔ Salida

```
{'edad': 26, 'ciudad': 'Bogota'}
```

```
1 precios = {"pan": 1.5, "leche": 2.0, "cafe": 3.5}
2
3 # Iterar por claves y valores al mismo tiempo con .items
4   ()
5 for producto, precio in precios.items():
6     print(f"El precio de {producto} es ${precio}")
```

↓ Salida del bucle

```
El precio de pan es $1.5
El precio de leche es $2.0
El precio de cafe es $3.5
```

Tabla Comparativa

Colección	Sintaxis	Ordenada	Mutable	Duplicados
Lista	[]	Sí	Sí	Sí
Tupla	()	Sí	No	Sí
Diccionario	{k:v}	Sí*	Sí	Claves No

i *A partir de Python 3.7+, los diccionarios mantienen el orden de inserción de los elementos de manera interna.

Mini-Proyecto: Agenda de Contactos

```
1 # Base de datos usando una lista de diccionarios
2 agenda = []
3
4 def agregar_contacto(nombre, telefono, email):
5     contacto = {
6         "nombre": nombre,
7         "telefono": telefono,
8         "email": email
9     }
10    agenda.append(contacto)
11    print(f"Contacto {nombre} guardado exitosamente.")
12
13 agregar_contacto("Luis", "555-1234", "luis@mail.com")
14 agregar_contacto("Maria", "555-5678", "maria@mail.com")
```

Mini-Proyecto (Continuación)

```
1 print("\n--- Lista de contactos ---")
2 for c in agenda:
3     print(f"Nombre: {c['nombre']} | Tel: {c['telefono']}")
4     )
```

➔ Salida final de la agenda

```
Contacto Luis guardado exitosamente.
Contacto Maria guardado exitosamente.
--- Lista de contactos ---
Nombre: Luis | Tel: 555-1234
Nombre: Maria | Tel: 555-5678
```

¿Qué aprendimos hoy?

- ✓ **Listas:** Secuencias mutables para manejo general de datos.
- ✓ **Tuplas:** Secuencias inmutables seguras y optimizadas para rendimiento.
- ✓ **Diccionarios:** Almacenamiento ágil mediante mapeo de claves y valores.
- ✓ **Buenas prácticas:** Seleccionar la estructura correcta según la mutabilidad y la necesidad de buscar elementos.

⌞/⌟ ¡Domina las colecciones y dominarás los datos!



¡Preguntas!

Gracias por su atención